

DS542 Deep Learning for Data Science

Spring 2025 Midterm Challenge

Bhuvan Shivalingaiah Gowda
U80010687

AI Disclosure

For this assignment, I used ChatGPT for:

- Refining explanations of hyperparameters, regularization techniques, and data augmentation strategies.
- Improving the clarity and readability of the write-up.
- Took suggestions for hyperparameter tuning like learning rate, weight decaying.

All code implementation, model selection, and training were done independently by me. AI was not used to write or modify the Python code.

Model Description

Brief Descriptions of First Two Models

1. SimpleCNN (Baseline Model)
 - A basic convolutional neural network (CNN) architecture.
 - Trained for 5 epochs with a learning rate of 0.1 and a batch size of 8.
 - Optimization: Not fine-tuned; serves as a basic benchmark.
 - Kaggle Accuracy: 1%
2. ResNet18
 - A deeper architecture with residual connections for improved feature extraction.
 - Trained for 10 epochs with a learning rate of 0.01 and a batch size of 64.
 - Performance: Significant improvement over SimpleCNN.
 - Kaggle Accuracy: 20%

Best Model: Fine-Tuned ResNet101

The best-performing model was ResNet101FineTune, a fine-tuned version of ResNet101.

- Why ResNet101?

Deeper than ResNet18, allowing for richer hierarchical feature extraction. Pretrained weights were leveraged for transfer learning to improve performance. Fine-tuning enabled the model to adapt specifically to CIFAR-100.

- Key Modifications

Modified Fully Connected Layer: Replaced the final layer with a new linear layer with 100 output neurons (for CIFAR-100 classification). Batch Normalization and Dropout added to improve generalization. Adaptive Pooling for handling variations in image size.

Hyperparameter Tuning

- Batch Size: Increased to 128 for better stability and GPU utilization.
- Learning Rate: Lowered to $5e-4$, as fine-tuning required a smaller LR to prevent overfitting on the new dataset.
- Epochs: Increased to 50, allowing the model to converge more effectively.
- Optimizer: AdamW, which handles weight decay better than standard Adam.
- Weight Decay: Implicitly handled through AdamW, preventing overfitting.

Regularization Techniques

- Dropout: Added to the fully connected layers to reduce overfitting.
- Weight Decay: Prevents large weight updates, improving generalization.
- Batch Normalization: Helps stabilize training and allows the network to learn more robust features.

Data Augmentation Strategy

Data augmentation was crucial in improving model robustness:

- Random Horizontal Flip ($p=0.5$) – Helps generalization.
- Random Crop with Padding – Simulates variations in object positioning.
- Color Jitter – Enhances robustness to lighting variations.
- Normalization – Standardized input to match pretrained model expectations.

Results Analysis

Final Kaggle Accuracy: 46%, the best among all tested models.

Strengths:

- Significantly outperformed SimpleCNN and ResNet18 due to fine-tuning.
- Handled CIFAR-100's diverse classes better than shallower architectures.

Weaknesses:

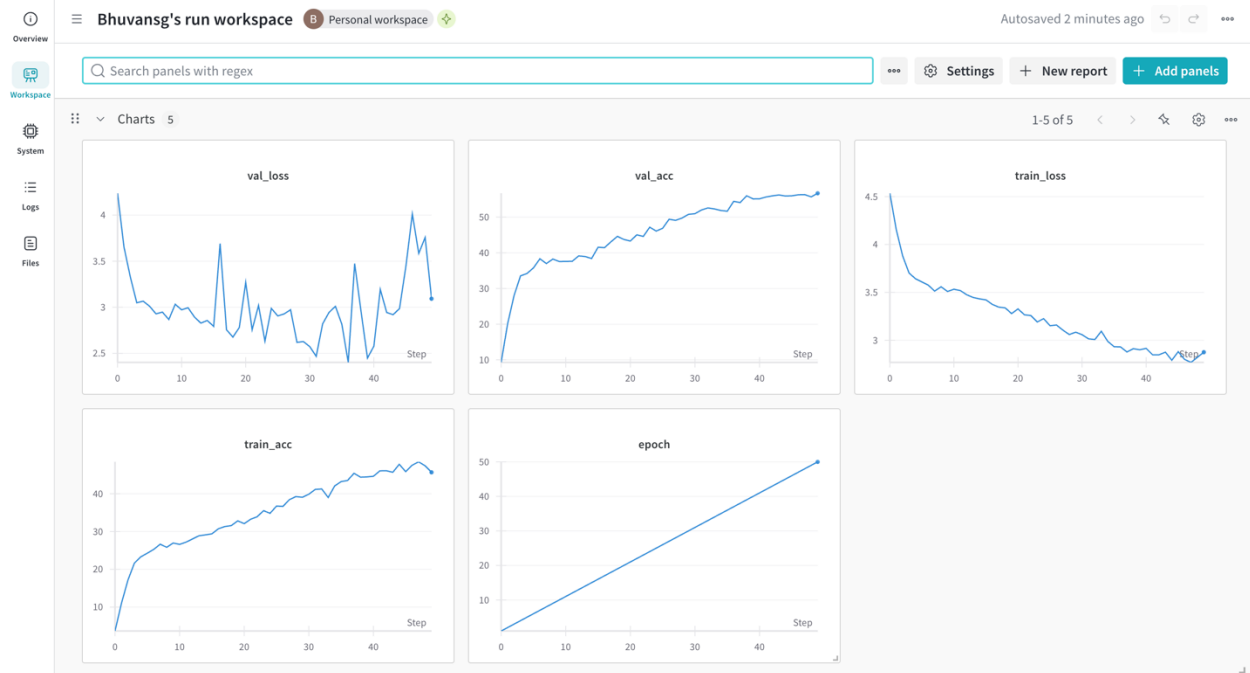
- Further improvement possible with additional hyperparameter tuning.
- Could explore stochastic depth for even better generalization.

Experiment Tracking Summary

All experiments were logged using Weights & Biases (WandB).

- Logged Metrics: Training and validation loss, training and validation accuracy.
- Comparison: Clearly showed ResNet101FineTune outperforming earlier models.

Best model –



All runs –

